Soprillo: Small, but Mechanized SAX

Arman Ahmed, Max Gross, Frédéric Lahaie-Bertrand

April 2026

Introduction

Mechanize a subset of the Semi-Axiomatic Sequent Calculus (SAX)

- **Goal 1**: Mechanize its proof theory (translations between G_3 and Sax, admissibility of cut, cut elimination, and subformula property)
- **Goal 2**: Mechanize its operational semantics and prove some of its properties (type preservation and progress) (Stretch)

Overview of the presentation:

- Fred: SAX and its proof theory
- Arman: Computational interpretation
- Max: Mechanization details

A Brief Run Through of History

The way we formalize intuitionistic logic can change its computational interpretation

Curry-Howard isomorphism

Herbelin introduces LJT based on the sequent calculus - explicit substitutions and concatenations

What happens when we replace non-invertible rules with axioms?

Semi-Axiomatic Sequent Calculus (SAX)

SAX arises from G_3 by replacing all **noninvertible** rules with axioms while leaving all the **invertible** rules unchanged

- Invertible: conclusion derivable $\Rightarrow$ premises derivable

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \supset B} \supset R$$

- Noninvertible: conclusion derivable $\not\Rightarrow$ premises derivable

$$\frac{\Gamma, A \supset B \vdash A \quad \Gamma, A \supset B, B \vdash C}{\Gamma, A \supset B \vdash C} \supset L \implies \frac{}{\Gamma, A, A \supset B \vdash B} \supset L^0$$

Semi-Axiomatic Sequent Calculus (SAX)

$$\frac{}{\Gamma, A \vdash A}\ \mathrm{Id}_A \qquad \frac{\Gamma \vdash A \quad \Gamma, A \vdash C}{\Gamma \vdash C}\ \mathrm{Cut}_A$$

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \supset B}\ \supset R \qquad \frac{}{\Gamma, A, A \supset B \vdash B}\ \supset L^0$$

$$\frac{}{\Gamma, A, B \vdash A \otimes B}\ \otimes R^0 \qquad \frac{\Gamma, A \otimes B, A, B \vdash C}{\Gamma, A \otimes B \vdash C}\ \otimes L$$

$$\frac{}{\Gamma \vdash 1}\ 1R^0 \qquad \frac{\Gamma, 1 \vdash C}{\Gamma, 1 \vdash C}\ 1L$$

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \,\&\, B}\ \&R \qquad \frac{}{\Gamma, A \,\&\, B \vdash A}\ \&L_1^0 \qquad \frac{}{\Gamma, A \,\&\, B \vdash B}\ \&L_2^0$$

$$\frac{}{\Gamma, A \vdash A \vee B}\ \vee R_1^0 \qquad \frac{}{\Gamma, B \vdash A \vee B}\ \vee R_2^0 \qquad \frac{\Gamma, A \vee B, A \vdash C \quad \Gamma, A \vee B, B \vdash C}{\Gamma, A \vee B \vdash C}\ \vee L$$

Cut-Elimination in SAX

Standard **cut-elimination fails**:

- Axioms introduce formulas that can't be decomposed
- No premises ⇒ no way to reduce the cut or to push it upwards

Example:

$$\dfrac{\dfrac{\dfrac{}{A, A \supset B \vdash B}\,{\supset}L^0 \quad \dfrac{}{B, B \supset C \vdash C}\,{\supset}L^0}{A, A \supset B, B \supset C \vdash C}\,\mathrm{Cut}_B}{A \supset B, B \supset C \vdash A \supset C}\,{\supset}R$$

Can't eliminate the cut on B... but the cut is **analytic**

- Analytic cut: cut formula is a subformula of the conclusion sequent
- Subformula property is not lost

Snips

Some cuts are allowed in SAX: **Snips**

- Snips are cuts where the cut formula is **eligible**, i.e. it arose as a subformula inside an axiom's principal formula
- Eligibility is tracked by labelling formula occurrences as eligible (*) propagating through the derivation
- Note: eligibility is a known **property**, not part of the syntax of sequents
- We also track irrelevant formulas (^{0}) that are never used (for cut elimination)

$$\dfrac{\dfrac{\dfrac{}{u^* : A, x : A \supset B, y^0 : B \supset C \vdash t^* : B}\ \supset L^0 \quad \dfrac{}{u^0 : A, x^0 : A \supset B, t^* : B, y : B \supset C \vdash w^* : C}\ \supset L^0}{u^* : A, x : A \supset B, y : B \supset C \vdash w^* : C}\ \mathrm{Snip}_B}{x : A \supset B, y : B \supset C \vdash z : A \supset C}\ \supset R$$

Two snips rules depending on which side contains the eligible occurrence:

$$\dfrac{\Gamma \vdash x^* : A \quad \Gamma, x : A \vdash z : C}{\Gamma \vdash z : C}\ Snip^1_A \qquad \dfrac{\Gamma \vdash x : A \quad \Gamma, x^* : A \vdash z : C}{\Gamma \vdash z : C}\ Snip^2_A$$

Labelled SAX

$$\frac{}{\Gamma^0,\ x : A \vdash y : A}\ \mathrm{Id}_A \qquad \frac{\Gamma \vdash x : A \quad \Gamma,\ x : A \vdash z : C}{\Gamma \vdash z : C}\ \mathrm{Cut}_A$$

$$\frac{\Gamma \vdash x^* : A \quad \Gamma,\ x : A \vdash z : C}{\Gamma \vdash z : C}\ \mathrm{Snip}^1_A \qquad \frac{\Gamma \vdash x : A \quad \Gamma,\ x^* : A \vdash z : C}{\Gamma \vdash z : C}\ \mathrm{Snip}^2_A$$

$$\frac{\Gamma,\ y : A \vdash z : B}{\Gamma \vdash x : A \supset B}\ \supset R \qquad \frac{}{\Gamma^0,\ y^* : A,\ x : A \supset B \vdash z^* : B}\ \supset L^0$$

$$\frac{}{\Gamma^0,\ y^* : A,\ z^* : B \vdash x : A \otimes B}\ \otimes R^0 \qquad \frac{\Gamma,\ x : A \otimes B,\ y : A,\ z : B \vdash w : C}{\Gamma,\ x : A \otimes B \vdash w : C}\ \otimes L$$

$$\frac{}{\Gamma^0 \vdash x : 1}\ 1R^0 \qquad \frac{\Gamma,\ x : 1 \vdash w : C}{\Gamma,\ x : 1 \vdash w : C}\ 1L$$

$$\frac{\Gamma \vdash y : A \quad \Gamma \vdash z : B}{\Gamma \vdash x : A \,\&\, B}\ \&R \qquad \frac{}{\Gamma^0,\ x : A \,\&\, B \vdash y^* : A}\ \&L^0_1 \qquad \frac{}{\Gamma^0,\ x : A \,\&\, B \vdash z^* : B}\ \&L^0_2$$

$$\frac{}{\Gamma^0,\ x^* : A \vdash z : A \vee B}\ \vee R^0_1 \qquad \frac{}{\Gamma^0,\ y^* : B \vdash z : A \vee B}\ \vee R^0_2$$

$$\frac{\Gamma,\ z : A \vee B,\ x : A \vdash w : C \quad \Gamma,\ z : A \vee B,\ y : B \vdash w : C}{\Gamma,\ z : A \vee B \vdash w : C}\ \vee L$$

Proof Theory: Main Results

Cut-free in SAX = no general cuts, snips permitted:

Theorem (Admissibility of Cut in SAX)

If there are cut-free derivations of $\Gamma \vdash x : A$ and $\Gamma, x : A \vdash z : C$, then there exists a cut-free derivation of $\Gamma \vdash z : C$.

Theorem (Cut Elimination in SAX)

If there is a derivation of $\Gamma \vdash x : A$, then there is a cut-free derivation of the same sequent.

Theorem (Subformula Property)

All formulas in a cut-free derivation of $\Gamma \vdash x : A$ are subformulas of formulas in Γ or A.

Also, why not:

Theorem (Translations between G3 and SAX)

$\Gamma \vdash A$ in G_3 iff $\Gamma \vdash A$ in SAX.

Proof of Cut Admissibility

Proof of cut admissibility is slightly more involved:

- Similar general structure: nested induction on the cut formula A and on the structure of the derivations $\mathcal{D}$ and $\mathcal{E}$
- Must track eligibility and irrelevance:
  - Eligible cut $\Rightarrow$ snip, irrelevant cut $\Rightarrow$ dropped
  - otherwise: reduce (principal) or propagate upward
- Eligibility (property) is not always preserved under these transformations $\Rightarrow$ direct proof doesn't work
- To ensure progress: if eligibility is lost, a cut on A is replaced by a cut on a strict subformula $B < A$, allowing recursion

$\Longrightarrow$ our proof of cut admissibility and cut elimination only currently covers a small subset of SAX (Id, $\supset L^0$, $\supset R$, and Snip rules)

$\Longrightarrow$ **Soprillo**

The Basics

Key idea: variables become addresses in shared memory; a sequent

$$x_1 : A_1, \ldots, x_n : A_n \vdash P :: (z : C)$$

types a process P that **reads** from $x_1, \ldots, x_n$ and **writes** to z.

Rules correspond to memory operations:

- **Cut** $\rightarrow$ allocate a fresh cell (a *future*): $x \leftarrow P \,;\, Q$
- **Axioms** $\rightarrow$ write; **Invertible rules** $\rightarrow$ read; **Identity** $\rightarrow$ copy $a\ b$

Example: $\supset L^0$ reads the continuation at x, passing argument y and destination z:

$$\frac{}{\Gamma,\, y^* : A,\, x : A \supset B \vdash x^R.\langle y, z\rangle :: (z^* : B)}\ \supset L^0$$

More on dynamics

Configurations - Long story short, in order to make the dynamics better represent the notion of shared memory, we want to represent multicuts, to do these we use configurations which are multisets of: $\texttt{thread}(a, P)$, $!\texttt{cell}(a, W)$, $\texttt{cell}(c, _)$

$$
\begin{array}{lcll}
\mathsf{proc}\ c\ (x \leftarrow P\ ;\ Q) & \longrightarrow & \mathsf{proc}\ a\ ([a/x]P), \mathsf{cell}\ a\ _, \mathsf{proc}\ c\ ([a/x]Q) & (a \text{ fresh}) \\
!\mathsf{cell}\ b\ W, \mathsf{proc}\ a\ (a^W \leftarrow b^R), \mathsf{cell}\ a\ _ & \longrightarrow & !\mathsf{cell}\ a\ W & \\
\mathsf{proc}\ a\ (\mathbf{case}\ a^W\ K), \mathsf{cell}\ a\ _ & \longrightarrow & !\mathsf{cell}\ a\ K & \\
!\mathsf{cell}\ a\ K, \mathsf{proc}\ c\ (a^R.V) & \longrightarrow & \mathsf{proc}\ c\ (V \triangleright K) & \\
\\
!\mathsf{cell}\ a\ V, \mathsf{proc}\ a\ (\mathbf{case}\ a^R\ K) & \longrightarrow & \mathsf{proc}\ a\ (V \triangleright K) & \\
\mathsf{proc}\ a\ (a.V), \mathsf{cell}\ a\ _ & \longrightarrow & !\mathsf{cell}\ a\ V &
\end{array}
$$

Properties: Preservation, Progress, Confluence, Termination.

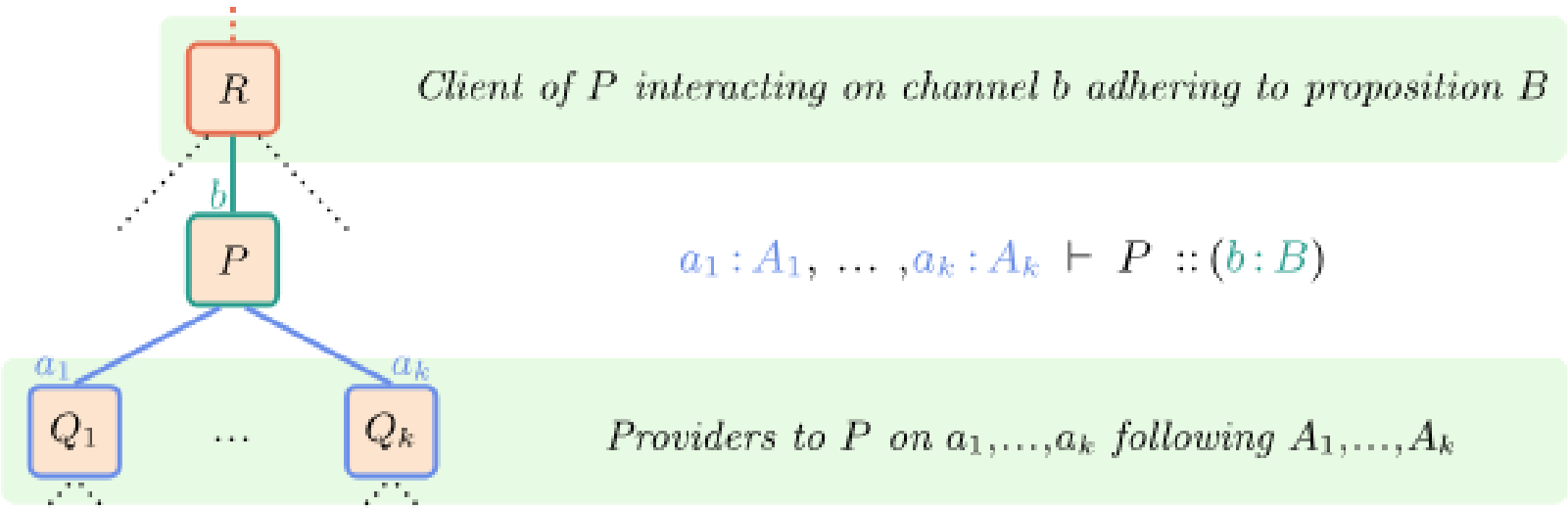
Message-Passing Interpretation

Comes from the adjoint formalization of SAX

Key Idea: Variables become channels, P provides a channel x

$$(x_1 : A^1) \ldots (x_n : A^n) \vdash P :: (x : A)$$

We add a mode $\sigma(m)$ that described the structural properties of a channel (a la adjoint logic), to go between we have shifts



Mechanizing SAX

- G_3 and Theorems 1 and 2 (cut elim. G_3) ✓ (Pfenning 2000) In LF, not Beluga
- SAX ✓ , Theorem 3 (translation) ✓
- miniSAX as a subset with $\supset$ only ✓) (possibility: extend to $\otimes$)
- Eligibility for snips as well-formedness predicate (a la linearity predicate) ✓
- Theorem 5 (cut admissibility) ✓, Theorem 6 (cut eligibility) should follow
- Remaining: Theorem 7 (Subformula property), configurations required for preservation, progress, etc.

- SAX ✓ , Theorem 3 (translation) ✓

```
LF hypS : o → type = ;
LF concS : o → type =
  | initialS : (hypS A → concS A)
  | &RS : concS A → concS B → concS (A & B)
  | &L01S : hypS (A & B) → concS A
  | &L02S : hypS (A & B) → concS B
  | ⊃RS : (hypS A → concS B) → concS (A ⊃ B)
  | ⊃L0S : hypS A → hypS (A ⊃ B) → concS B
  | ∨R01S : hypS A → concS (A ∨ B)
  | ∨R02S : hypS B → concS (A ∨ B)
  | ∨LS : (hypS A → concS C) → (hypS B → concS C) → (hypS (A ∨ B)
→ concS C)
  | cutS : concS A → (hypS A → concS C) → concS C
  | ⊗R0S : hypS A → hypS B → concS (A ⊗ B)
  | ⊗LS : (hypS A → hypS B → concS C) → (hypS (A ⊗ B) → concS C)
  | unitRS : concS unit
  | unitLS : concS C → (hypS unit → concS C)
;
rec seq2sax :  (A : [⊢ o]) rel-ctx-S [Γ] [Γ′] → [Γ ⊢ conc A[]] → [Γ′ ⊢
concS A[]] = ...
```

- miniSAX as a subset with $\supset$ only ✓) (possibility: extend to $\otimes$)

```
LF mhypS : mo → type = ;
LF mconcS : mo → type =
  | minitialS : (mhypS A → mconcS A)
  | m⊃RS : (mhypS A → mconcS B) → mconcS (A ⊃ B)
  | m⊃LOS : mhypS A → mhypS (A ⊃ B) → cmoncS B
  | mcutS : mconcS A → (mhypS A → mconcS C) → mconcS C
;
```

- miniSAX as a subset with $\supset$ only ✓) (possibility: extend to $\otimes$)

```
LF mhypS : mo → type = ;
LF mconcS : mo → type =
  | minitialS : (mhypS A → mconcS A)
  | m⊃RS : (mhypS A → mconcS B) → mconcS (A ⊃ B)
  | m⊃LOS : mhypS A → mhypS (A ⊃ B) → cmoncS B
  | mcutS : mconcS A → (mhypS A → mconcS C) → mconcS C
;
```

Require some way to discuss snips (eligibility) and get rid of `mcutS` for cut admissability...

- Eligibility for snips as well-formedness predicate (a la linearity predicate) ✓

```
LF mlhypS : mo → type = ;
LF melig :  mlhypS A → type = ;

LF mlconcS* :  mo → type =
  | m⊃lLOS : {h1 :  mlhypS A} {h2
:  mlhypS (A ⊃ B)}
    melig h1 → mlconcS* B

LF mlconcS : mo → type =
  | mlinitialS : {h :  mlhypS A}
→ mlconcS A
  | mlsnip1S : mlconcS* A →
(mlhypS A → mlconcS C) → mlconcS
C
  | mlsnip2S : mlconcS A → ({h :
mlhypS A} melig h → mlconcS C) →
mlconcS C
  | m⊃lRS : (mlhypS A → mlconcS
B) → mlconcS (A ⊃ B)
  | mlconcS/* :  mlconcS* A →
mlconcS A
;
```

$$\frac{}{\Gamma^0, y^* : A, x : A \supset B \vdash z^* : B} \supset L^0$$

$$\frac{\Gamma \vdash x^* : A \quad \Gamma, x : A \vdash y : C}{\Gamma \vdash z : C} \mathrm{Snip}^1_A$$

$$\frac{\Gamma \vdash x : A \quad \Gamma, x^* : A \vdash y : C}{\Gamma \vdash z : C} \mathrm{Snip}^1_A$$

- Theorem 5 (cut admissibility) ✓, Theorem 6 (cut eligibility) should follow

```
rec mcut-admis : {Γ: mlctx} {A:
[⊢ mo]}
  [Γ ⊢ mlconcS A[]]
  → [Γ, x:mlhypS A[] ⊢ mlconcS
C[]]
  → [Γ ⊢ mlconcS C[]] =

/ total 1 /

mlam Γ  mlam A  fn D  fn E
case (D, E) of

  ...

  | ([Γ ⊢ D], [Γ, x : mlhypS A[]
⊢ m⊃lRS (\h. C)])

    let [Γ, z : mlhypS _ ⊢ F] =
mcut-admis _ _
      [Γ, z : mlhypS _ ⊢ D[..]]
      [Γ, z : mlhypS _, x :
mlhypS A[] ⊢ C[..,z]]
    in

    [Γ ⊢ m⊃lRS (\h. F[..,h])]

  ...
:
```

$$\dfrac{\overset{\mathcal{D}}{\Gamma \vdash x : A} \quad \dfrac{\overset{\mathcal{E}}{\Gamma, x : A, z_1 : C_1 \vdash z_2 : C_2}}{\Gamma, x : A \vdash z : C_1 \supset C_2}\supset R}{\Gamma \vdash C_1 \supset C_2}\,\mathrm{Cut}_A$$

$$\Longrightarrow$$

$$\dfrac{\dfrac{\overset{\mathcal{D} + \{z_0^1 : C_1\}}{\Gamma, z_1^0 : C_1 \vdash x : A} \quad \overset{\mathcal{E}_0}{\Gamma, x : A, z_1 : C_1 \vdash z_2 : C_2}}{\Gamma, z_1 : C_1 \vdash z_2 : C_2}\,IH}{\Gamma \vdash z : C_1 \supset C_2}\supset R$$

- Remaining: Theorem 7 (Subformula property), configurations required for preservation, progress, etc.
- Theorem 7: All formulas in a cut-free derivation of $\Gamma \vdash x : A$ are subformulas of formulas in Γ or A. Unsure how to formalize this.
- Configurations are done on multisets, which prove challenging to work with in Beluga. Can we treat them as ordered lists?

Related Work

- We don't know of any other mechanizations of SAX itself, nor its proof theoretic properties
- DeYoung et al. present SAX and the meta-theoretic properties that we mechanize
- Francalaza et al. Presents an implementation of the message-passing interpretation of SAX in Go.
- Zhang et al. gives a mechanization of logical relations for message passing protocols in Coq.

Timeline

- Mechanize SAX and Soprillo ✓
- Mechanize translation to G_3 ✓
- Mechanize labelled SAX, Soprillo ✓
- Mechanize cut admissibility for Soprillo ✓
- Mechanize cut elimination - April 12th
- Mechanize shared memory interpretation (Stretch)
- Mechanize type preservation, confluence, Progress (Stretch)
- Finish write up - mid April